



# PARALLEL SAMPLE SORT

a hybrid Open MPI + OMP implementation  
and Native and Containerized benchmarking

# PARALLEL SAMPLE SORT

## ingredients



N unsorted keys (64 unsigned integers)



P processors (Open MPI))



T threads per process (OMP)

# PARALLEL SAMPLE SORT

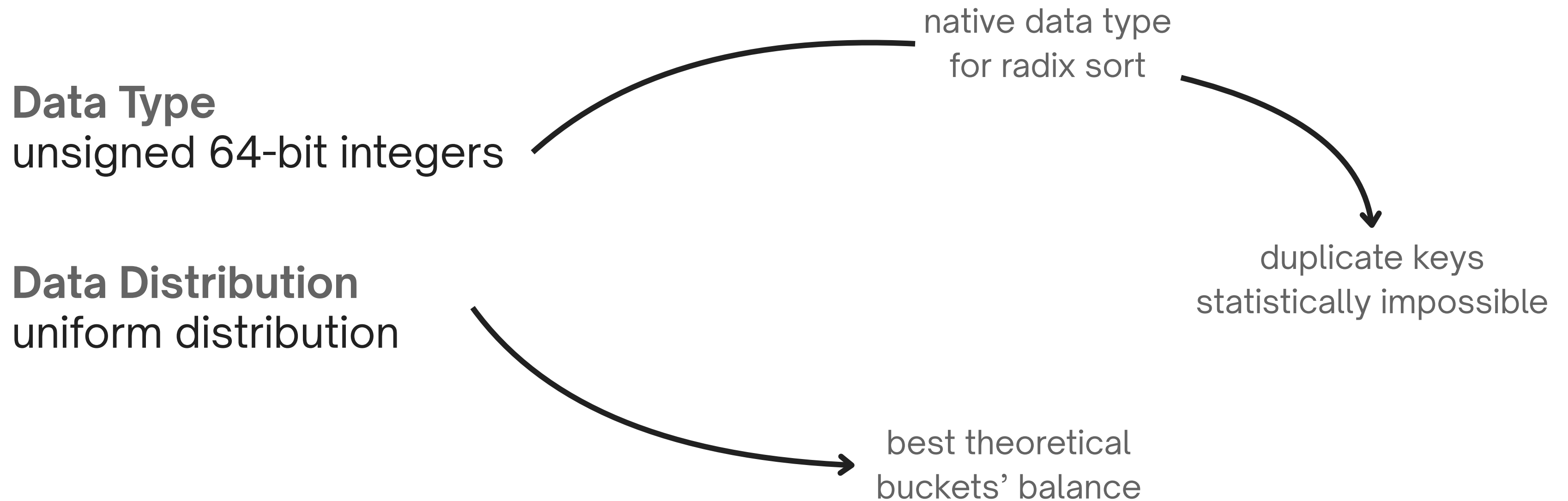
## procedure

1. **Local Generation** ( $N/P$  keys per process)
2. **Local Sort** (leveraging  $T$  threads)
3. **Local Regular Sampling**
4. **Global Pivot Selection** (on process #0)
5. **Local Partitioning** (wrt to the selected pivots)
6. **Global Buckets Exchange** (all to all)
7. **Local K-Way Merge** (one process, one pivot,  $P$  buckets)



# PARALLEL SAMPLE SORT

## assumptions



# PARALLEL SAMPLE SORT

local sort

**Quick Sort**

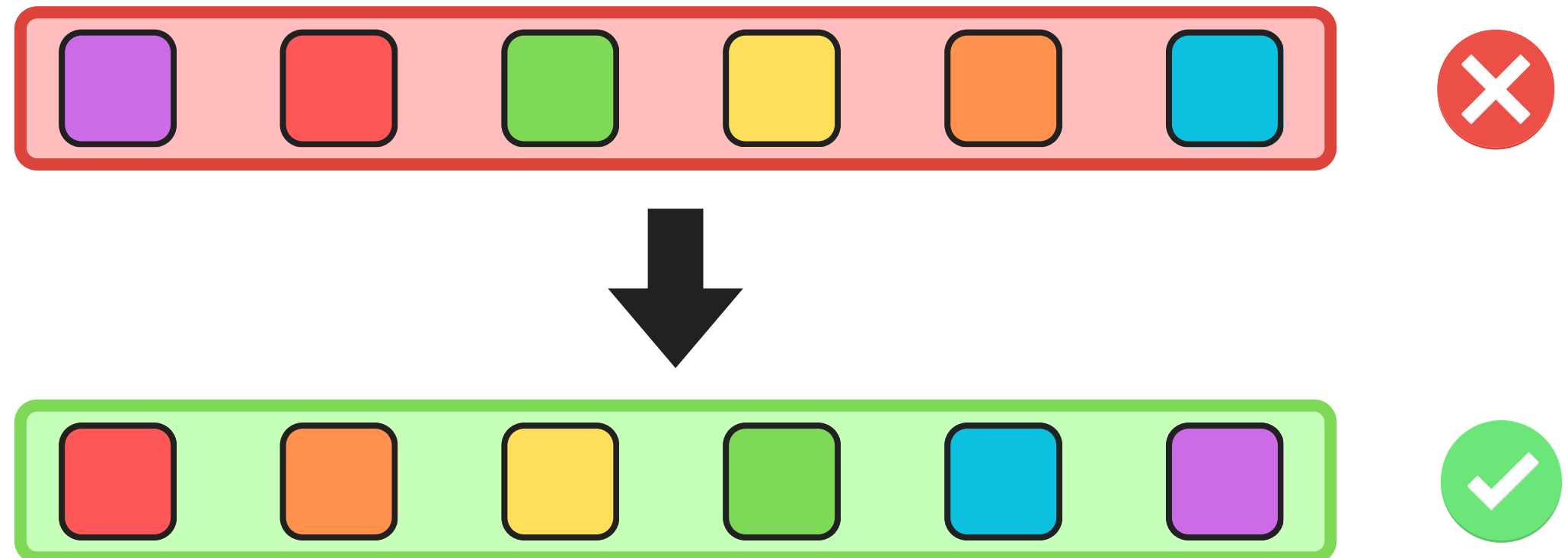
given  $n$  keys

$$O(n \log(n))$$

**Radix Sort**

given  $n$  keys, and digit size  $d$

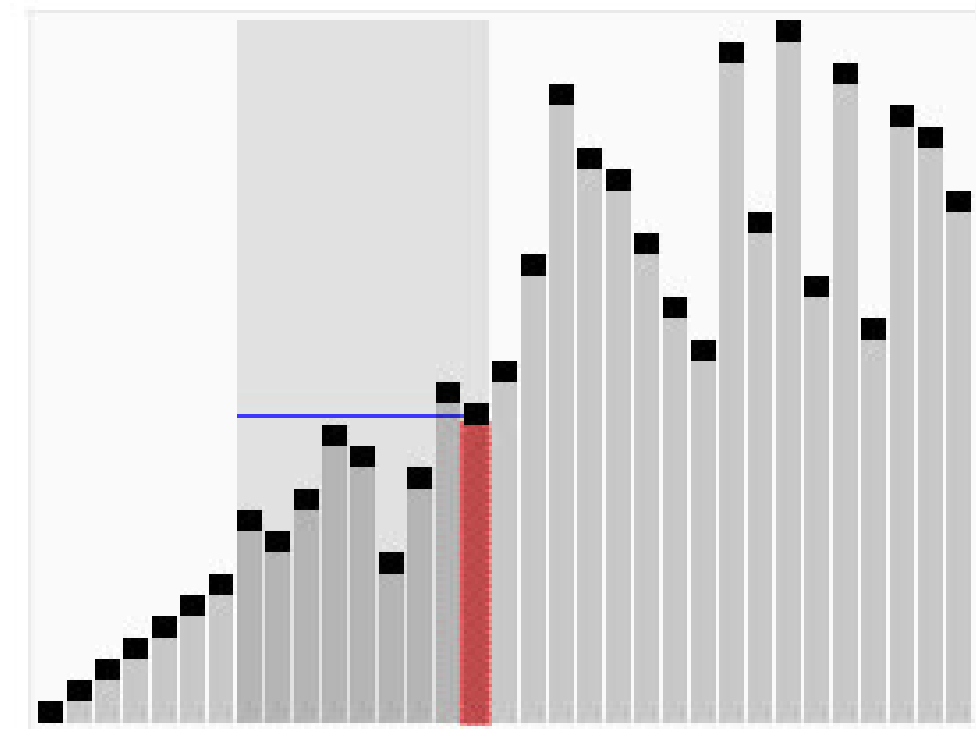
$$O(dn)$$



# QUICK SORT

details

- Recursive Approach
- OMP Tasks
- Serial Cutoff (at 25000 elements)
- Insertion Sort Cutoff (at 8 elements)
- MITM Pivot Selection



source Wikipedia

# RADIX SORT

## details

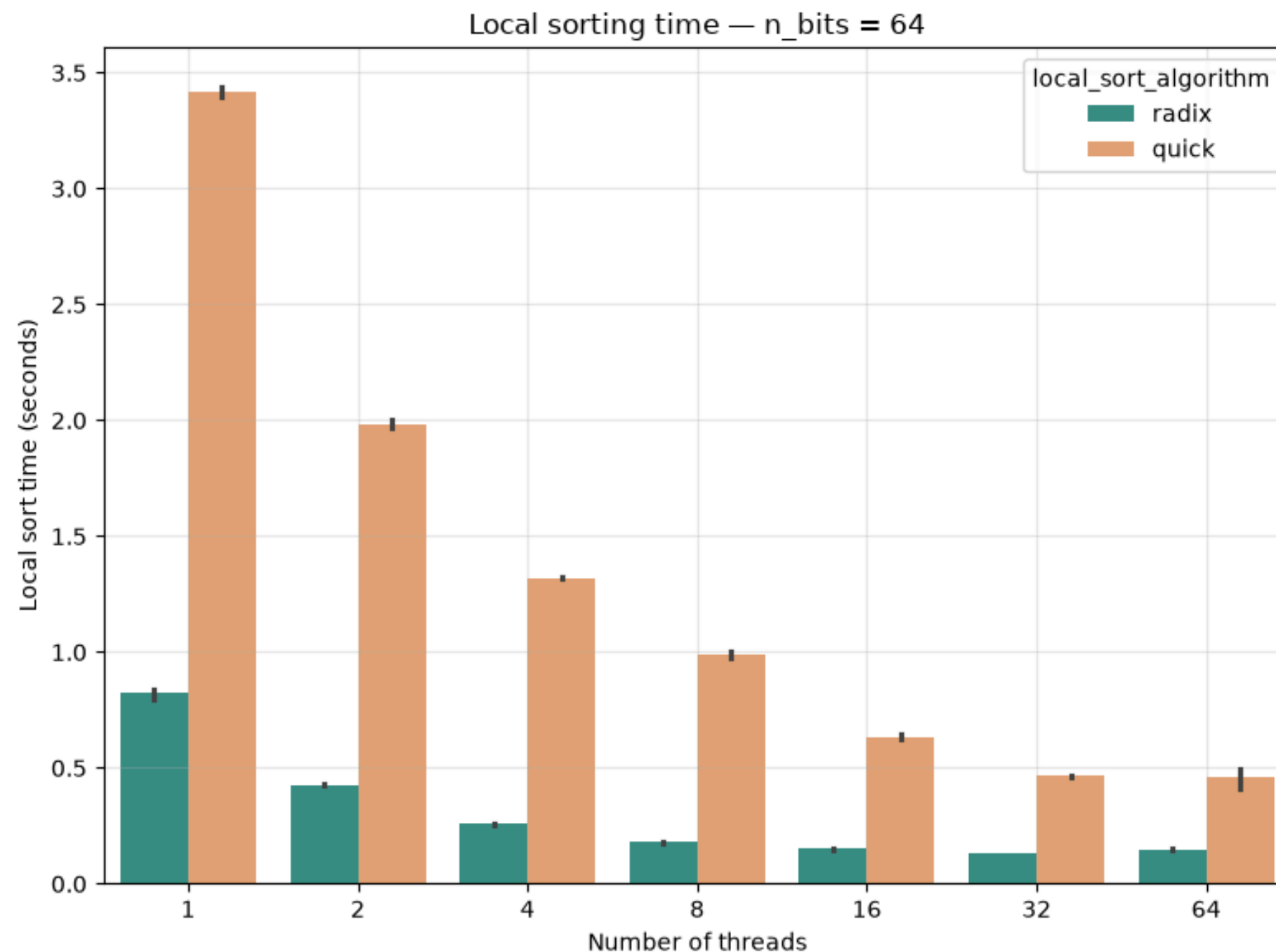
- Stable Counting Sort based
- Digit Sorting Parallelized
- Posix mem\_align for histograms
- Optimal Digit Size 8

| 1°<br>PASSO |   | 2°<br>PASSO |   | 3°<br>PASSO |   | 4°<br>PASSO |   |      |
|-------------|---|-------------|---|-------------|---|-------------|---|------|
| 253         |   | 10          |   | 5           |   | 5           |   | 5    |
| 346         |   | 253         |   | 10          |   | 10          |   | 10   |
| 1034        |   | 1034        |   | 127         |   | 1034        |   | 127  |
| 10          | → | 5           | → | 1034        | → | 127         | → | 253  |
| 5           |   | 346         |   | 346         |   | 253         |   | 346  |
| 127         |   | 127         |   | 253         |   | 346         |   | 1034 |

source Wikipedia

# PARALLEL SAMPLE SORT

## local sort



- nkeys =  $5 * 10^7$
- nranks = 1
- distribution = uniform

Radix sort clearly outperform quick sort.

Probable causes:

- radix's ideal configuration
- radix's OMP better implementation

\*All other experiments are shown in the report. Refere to it for detailed results. This is meant only as 15 minutes presentation.

# PARALLEL SAMPLE SORT

k-way merge

## Basic k-way Merge

given  $n$  keys, and  $k$  data streams

$$O(nk)$$

## Bin k-way Merge

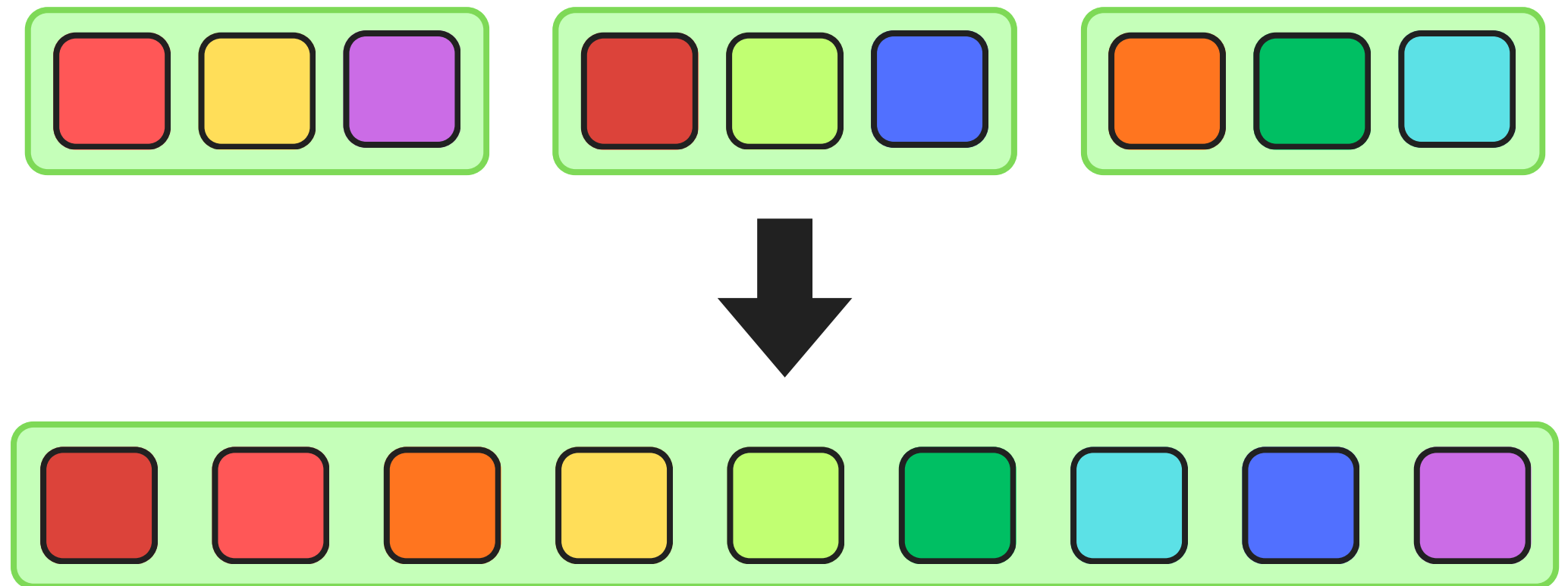
given  $n$  keys, and  $k$  data streams

$$O(n \log(k))$$

## Heap k-way Merge

given  $n$  keys, and  $k$  data streams

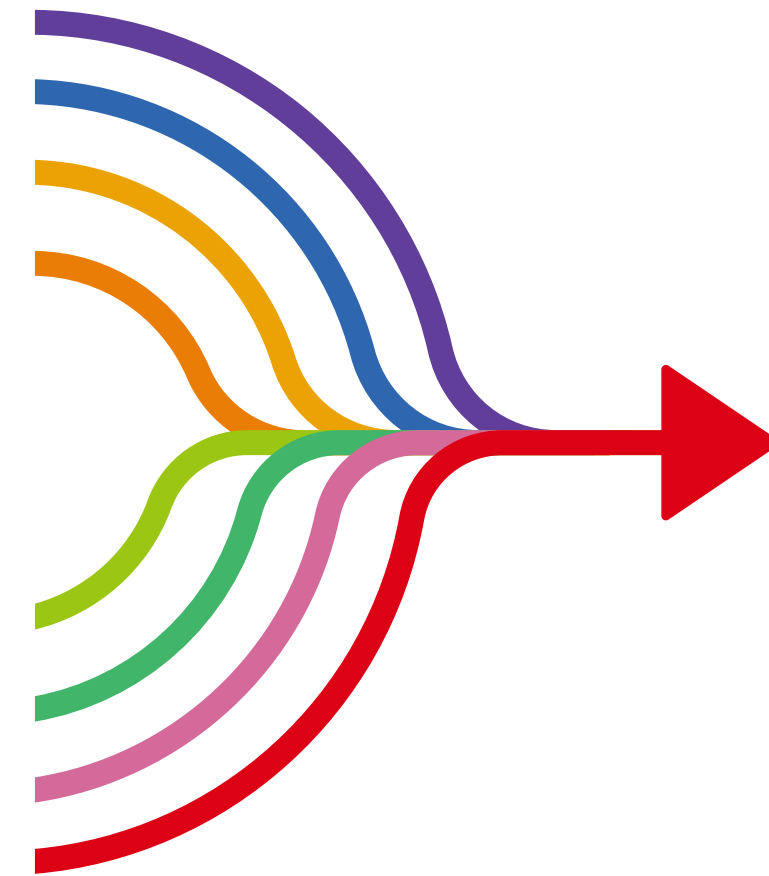
$$O(n \log(k))$$



# BASIC K-WAY MERGE

details

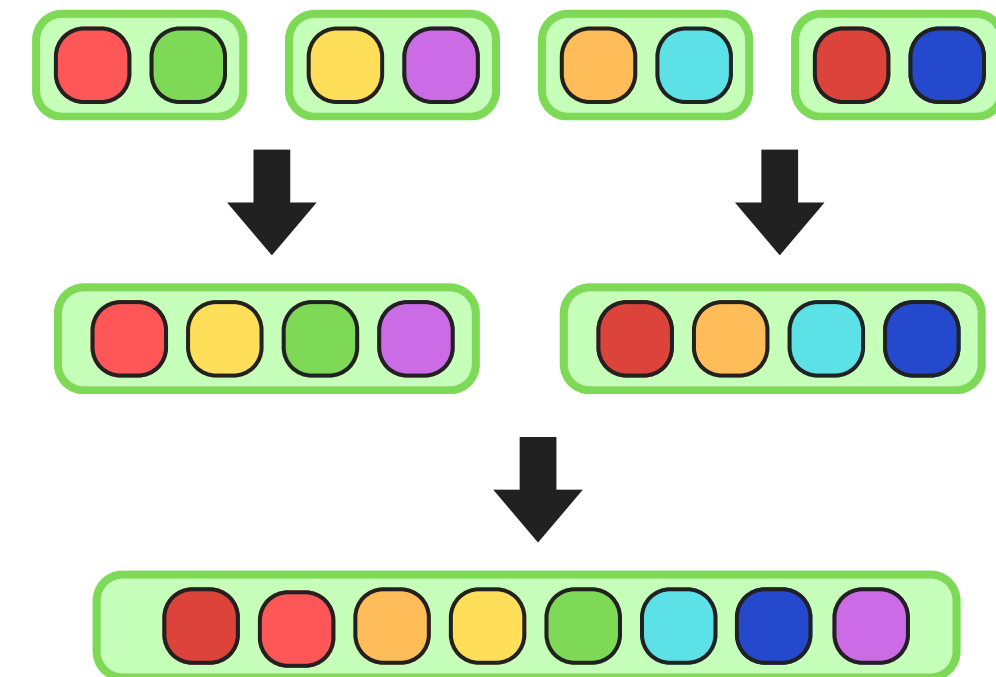
1. Scan the heads of the sorted k input streams
2. Select the minimum among them
3. Add to the output stream
4. Repeat until one stream is left



# BINARY K-WAY MERGE

details

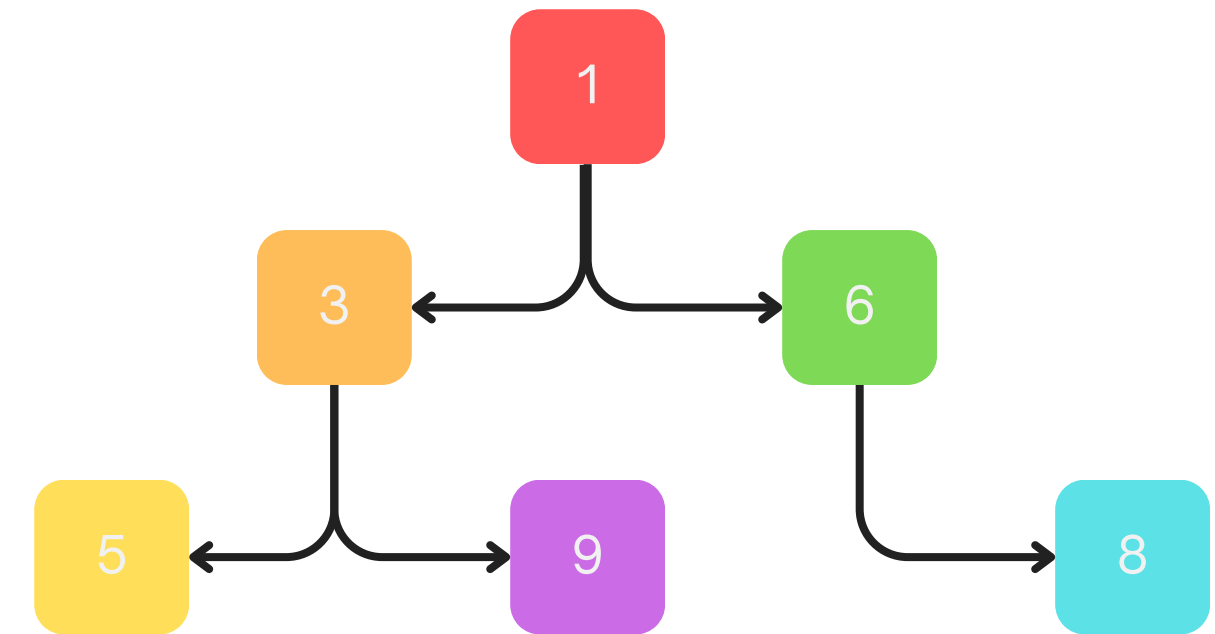
1. Pair input streams (0, 1) (2, 3) ...
2. Use basic 2-way merge on the pairs of streams
3. Repeat until one stream is left (treat the resulting output stream as input streams and pair them again)



# HEAP K-WAY MERGE

details

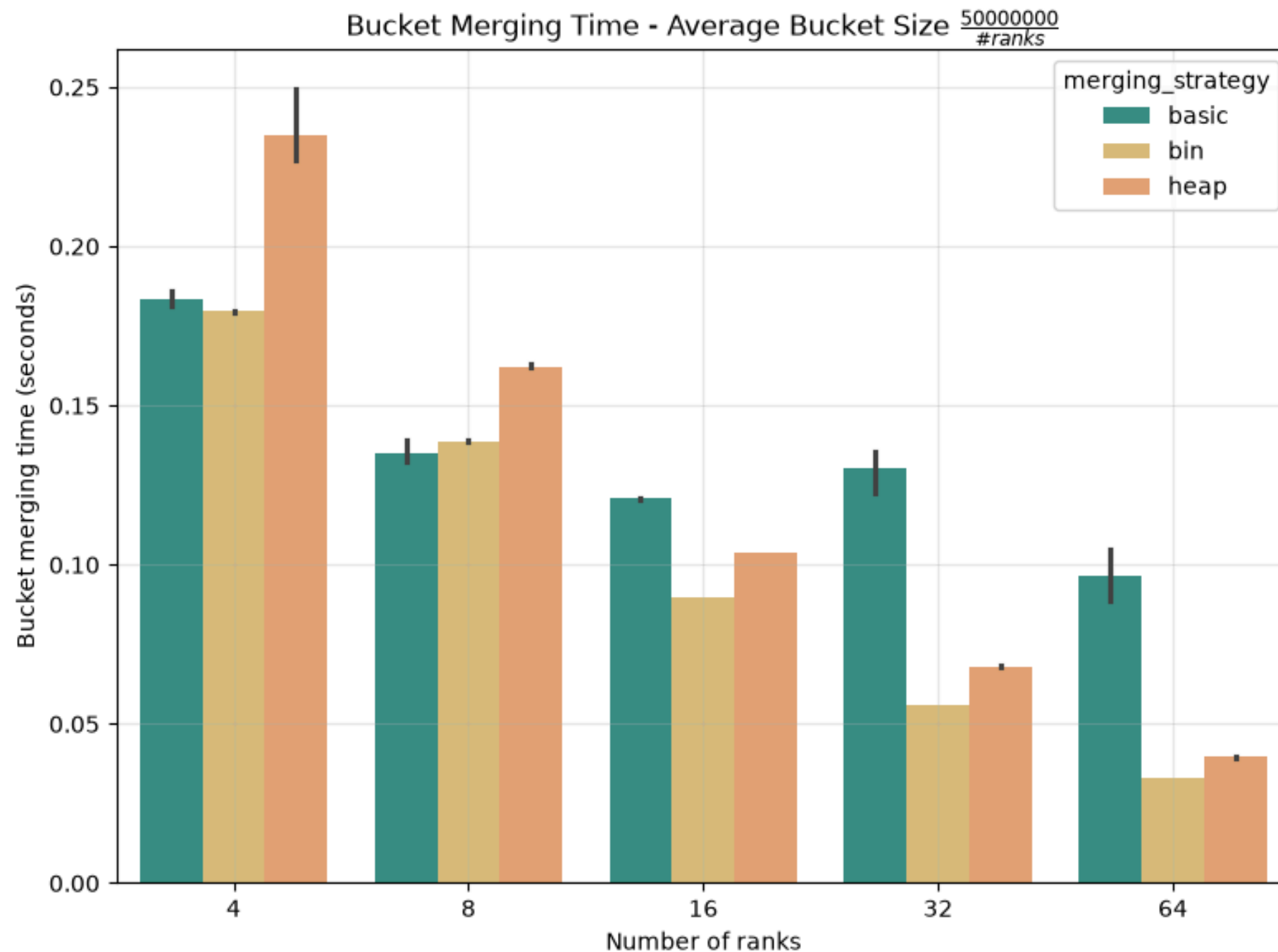
1. Construct a min-heap with the initial elements of each stream
2. Pop the root from the heap and add the following minimum from the stream that contained the root
3. Repeat until the heap is empty



# PARALLEL SAMPLE SORT

## k-way merge

13



- $nkeys = 5 * 10^7$
- $nthreads = 1$
- distribution = uniform

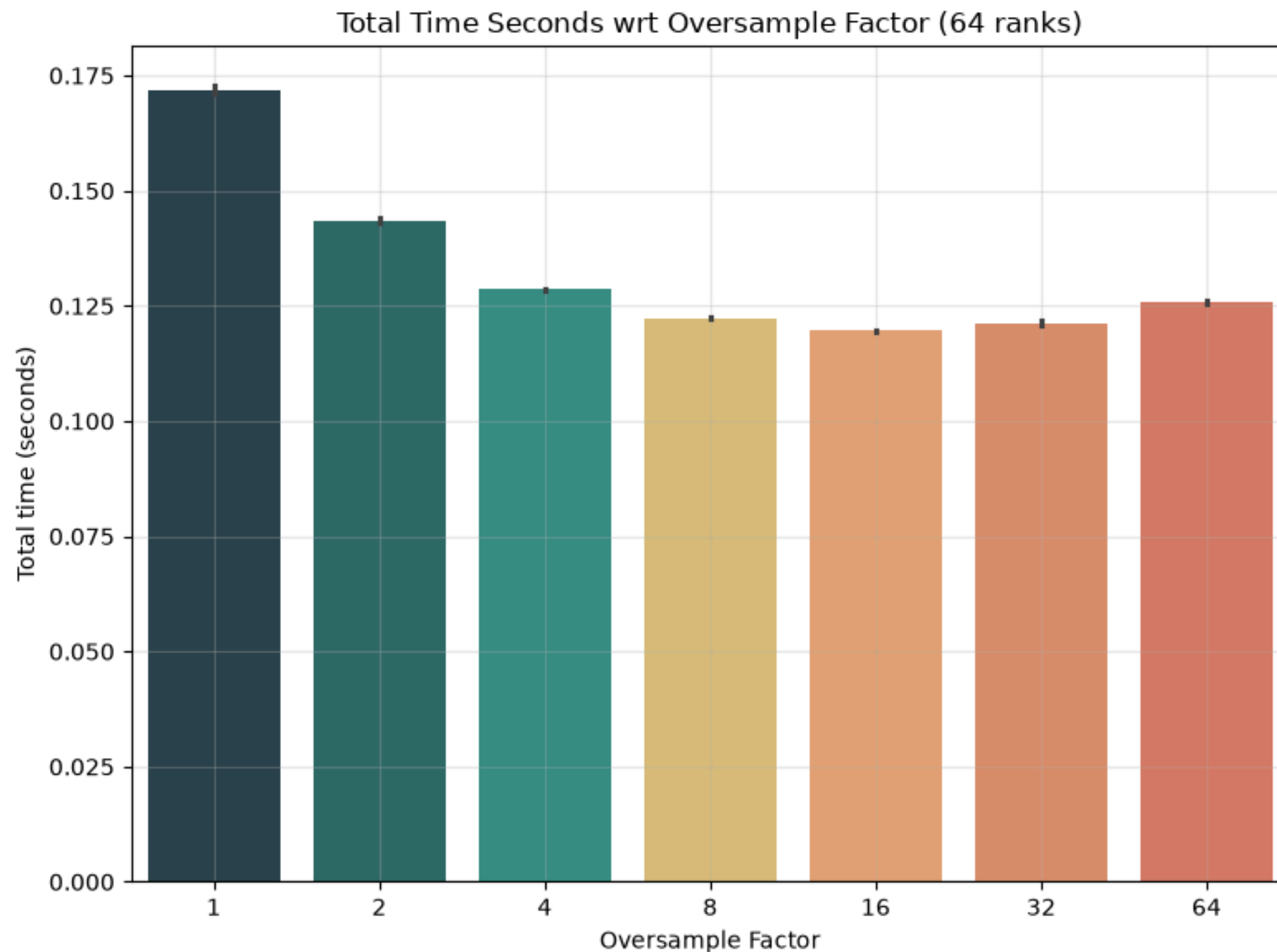
The binary strategy is the most robust among all. It has the same theoretical computational bound of the heap's one but it has less overhead due to no structure maintenance.

\*All other experiments are shown in the report. Refere to it for detailed results. This is meant only as 15 minutes presentation.

# PARALLEL SAMPLE SORT

## oversample factor

14



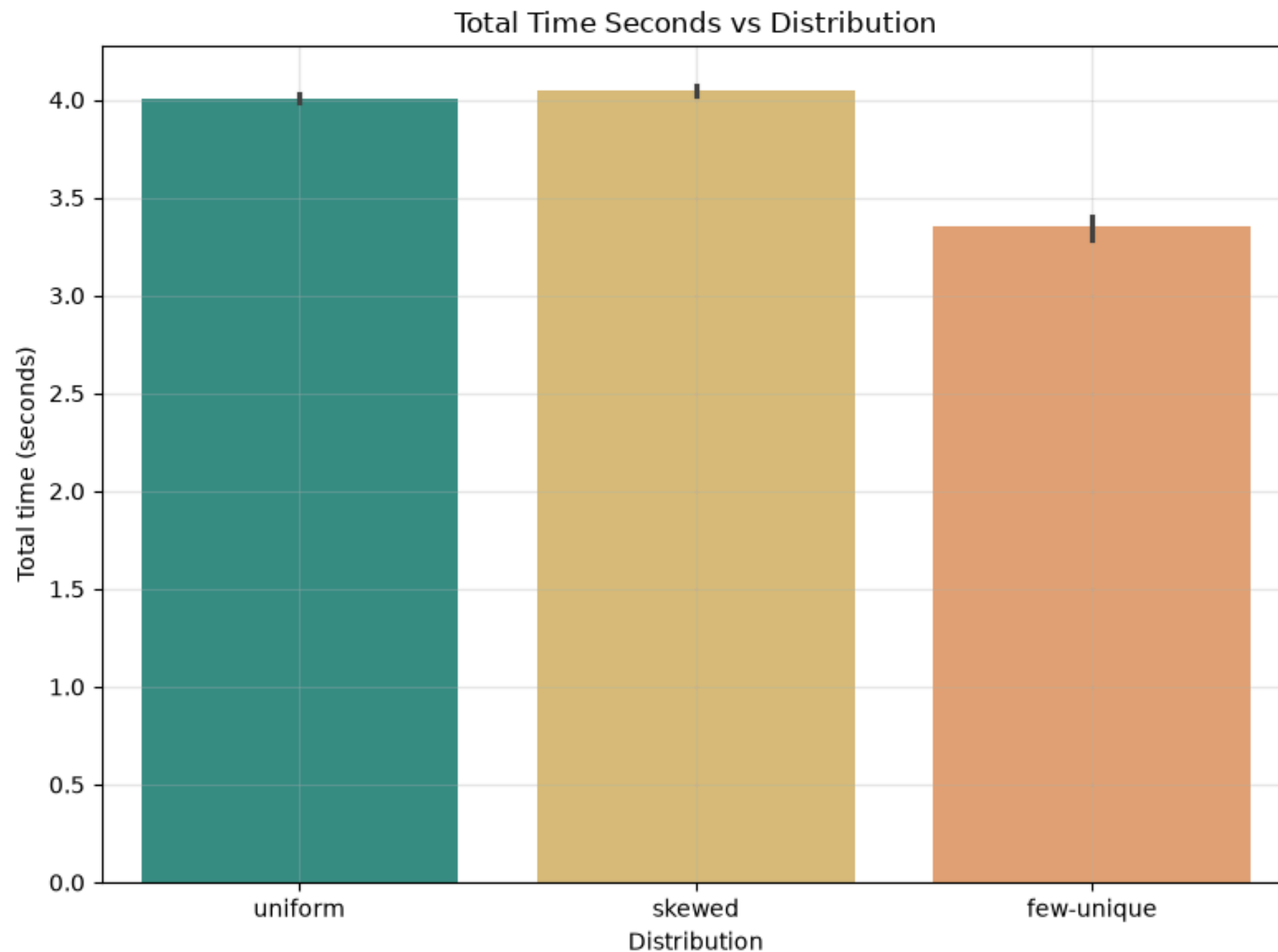
- $nkeys = 5 * 10^7$
- $nranks = 64$
- $nthreads = 1$
- distribution = uniform

The experiment has been conducted on a similar configuration with 8 ranks and 8 threads per rank. Similar results have been obtained.

\*All other experiments are shown in the report. Refere to it for detailed results. This is meant only as 15 minutes presentation.

# PARALLEL SAMPLE SORT

## skewed distribution



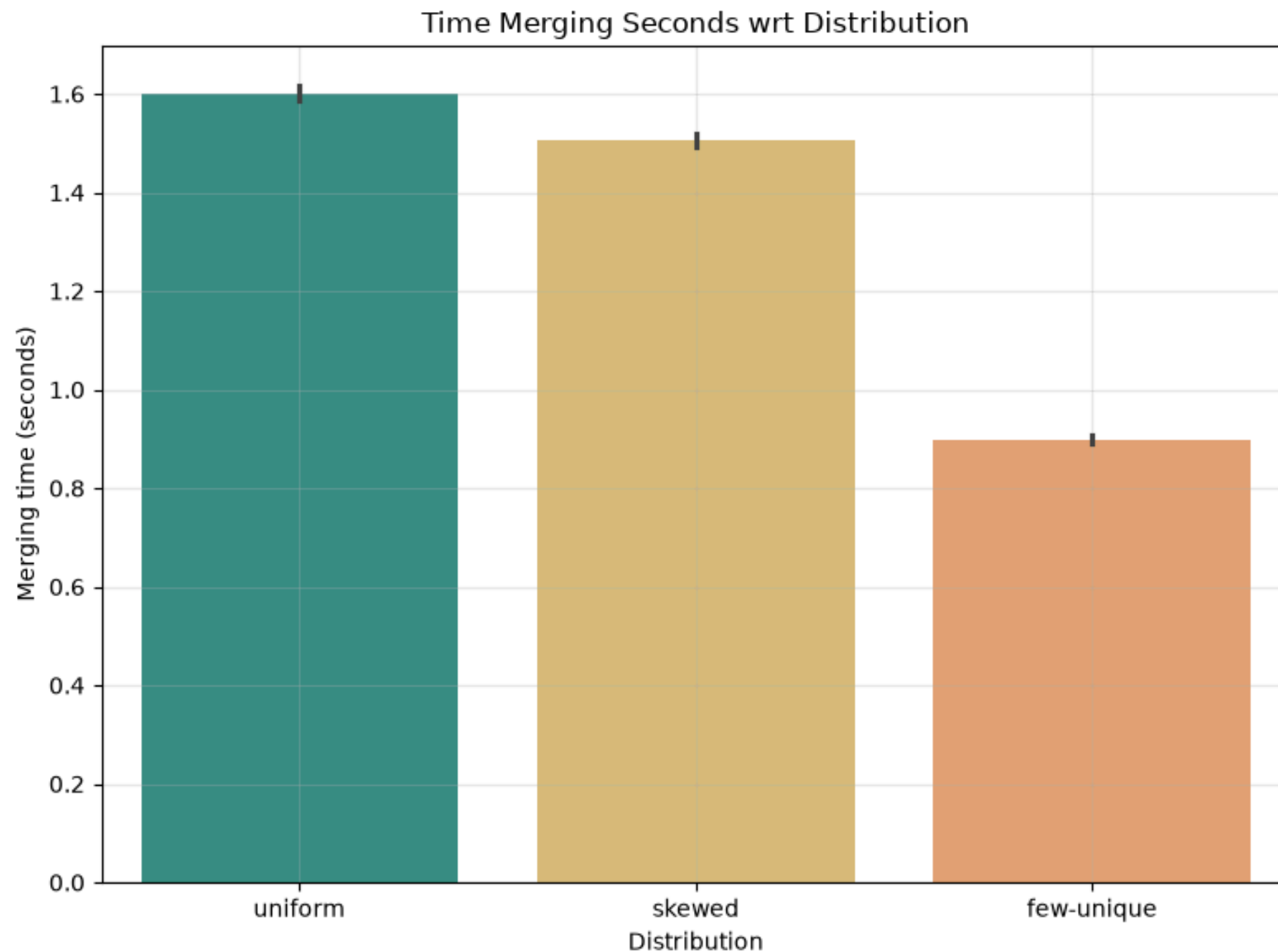
- nkeys =  $10^9$
- nranks = 16
- nthreads = 4
- distribution = uniform
- merging\_strategy = bin
- sorting = radix

Why this strange results?

\*All other experiments are shown in the report. Refere to it for detailed results. This is meant only as 15 minutes presentation.

# PARALLEL SAMPLE SORT

## skewed distribution



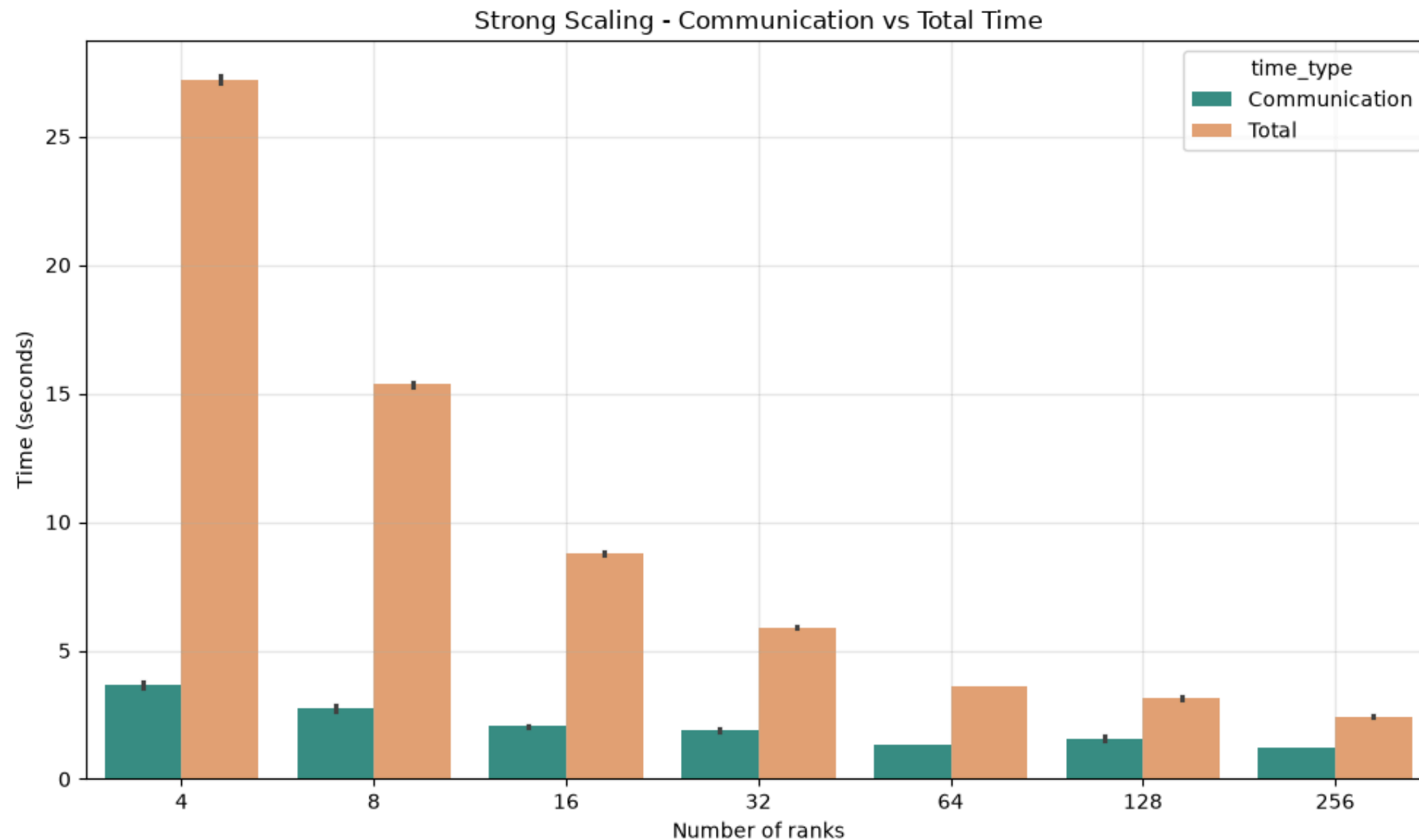
The merging time is the main relevant factor. As a consequence of the non-uniform distribution, a significant number of destination buckets can remain empty. This lead to quicker overall execution.

\*All other experiments are shown in the report. Refere to it for detailed results. This is meant only as 15 minutes presentation.

# STRONG SCALING

total and communication time

17



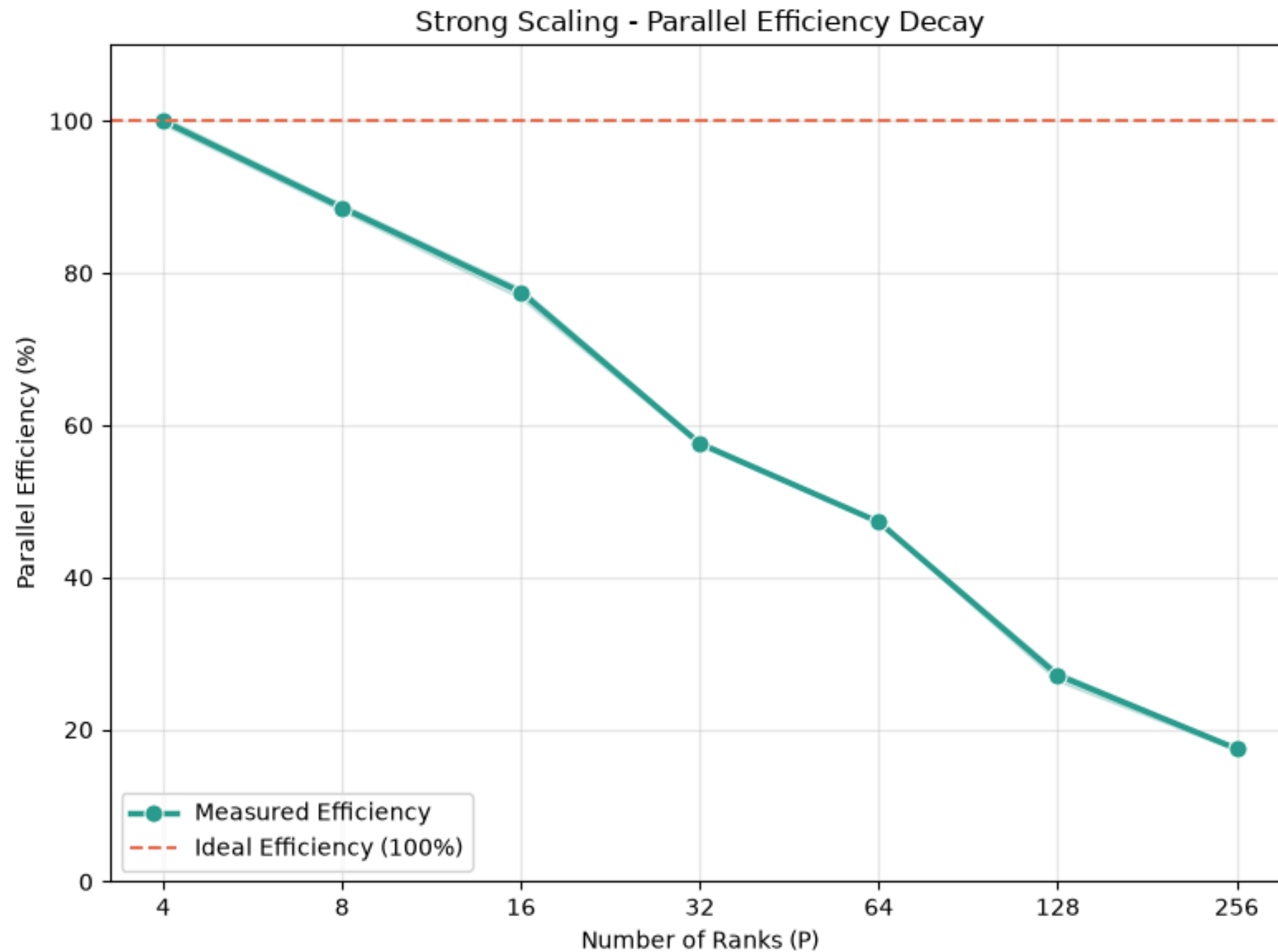
- nkeys =  $10^9$
- nthreads = 1
- distribution = uniform
- merging\_strategy = bin
- sorting = radix

The execution time is high because only one thread is used.

\*All other experiments are shown in the report. Refere to it for detailed results. This is meant only as 15 minutes presentation.

# STRONG SCALING

parallel efficiency decay



$$E_N = \frac{S_N}{N} = \frac{T_1}{NT_N}$$

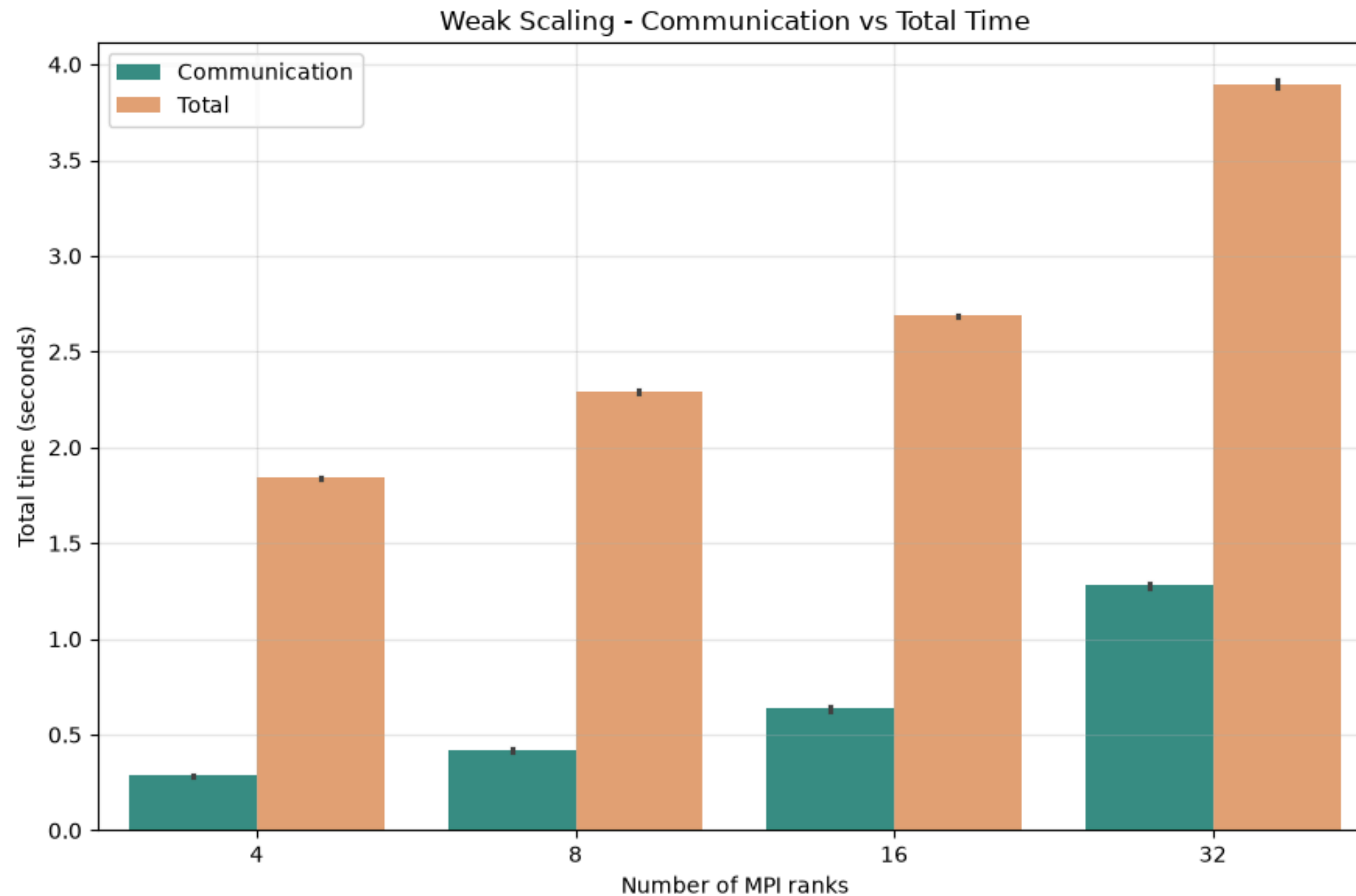
$$E_N^{(4)} = \frac{T_4}{NT_N}$$

$$E_N \approx 4E_N^4$$

\*All other experiments are shown in the report. Refere to it for detailed results. This is meant only as 15 minutes presentation.

# WEAK SCALING

## total and communication time

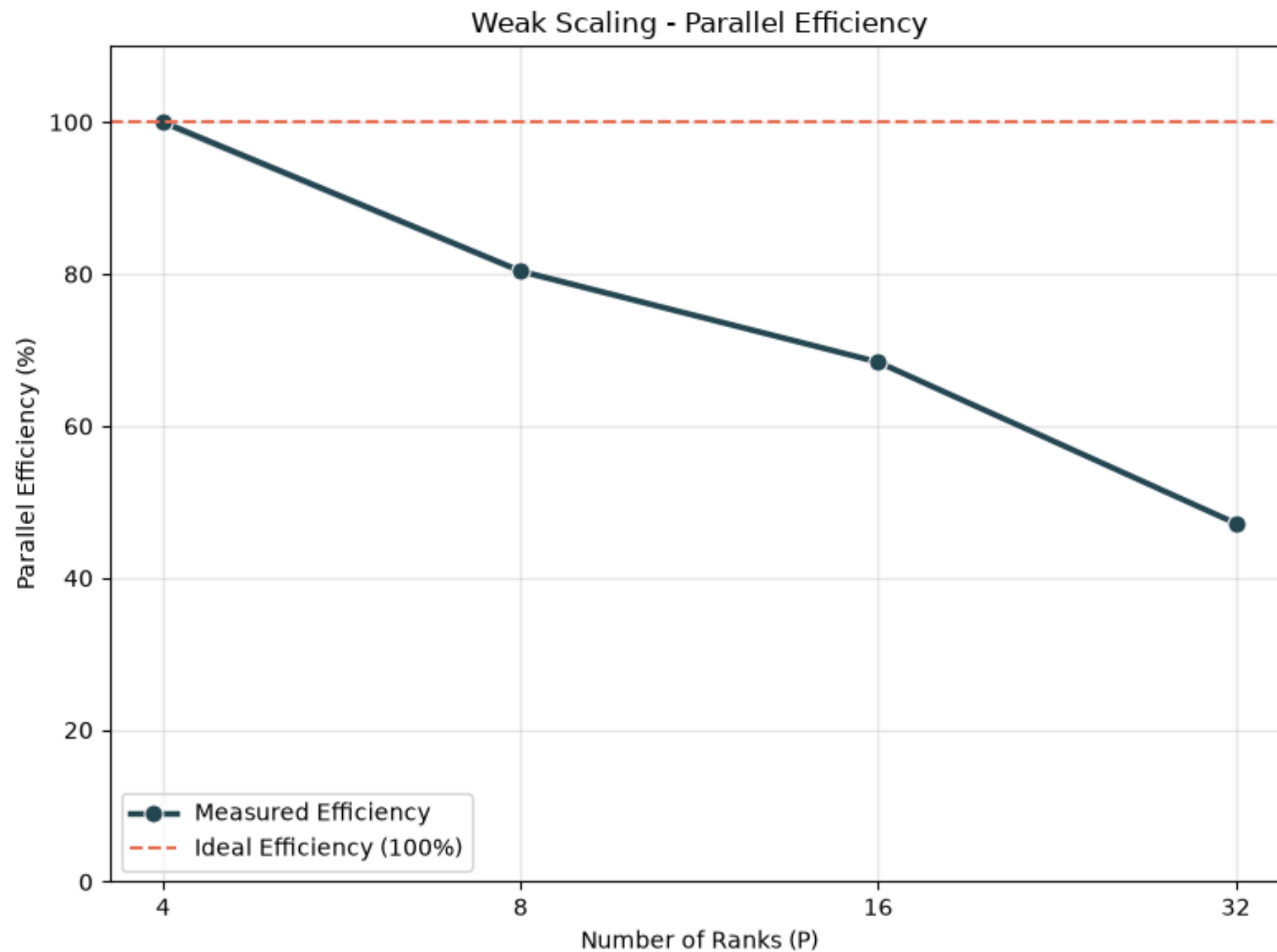


- $nkeys/nranks = 2 \times 10^7$
- $nthreads = 4$
- $distribution = uniform$
- $merging\_strategy = bin$
- $sorting = radix$

\*All other experiments are shown in the report. Refere to it for detailed results. This is meant only as 15 minutes presentation.

# WEAK SCALING

parallel efficiency decay

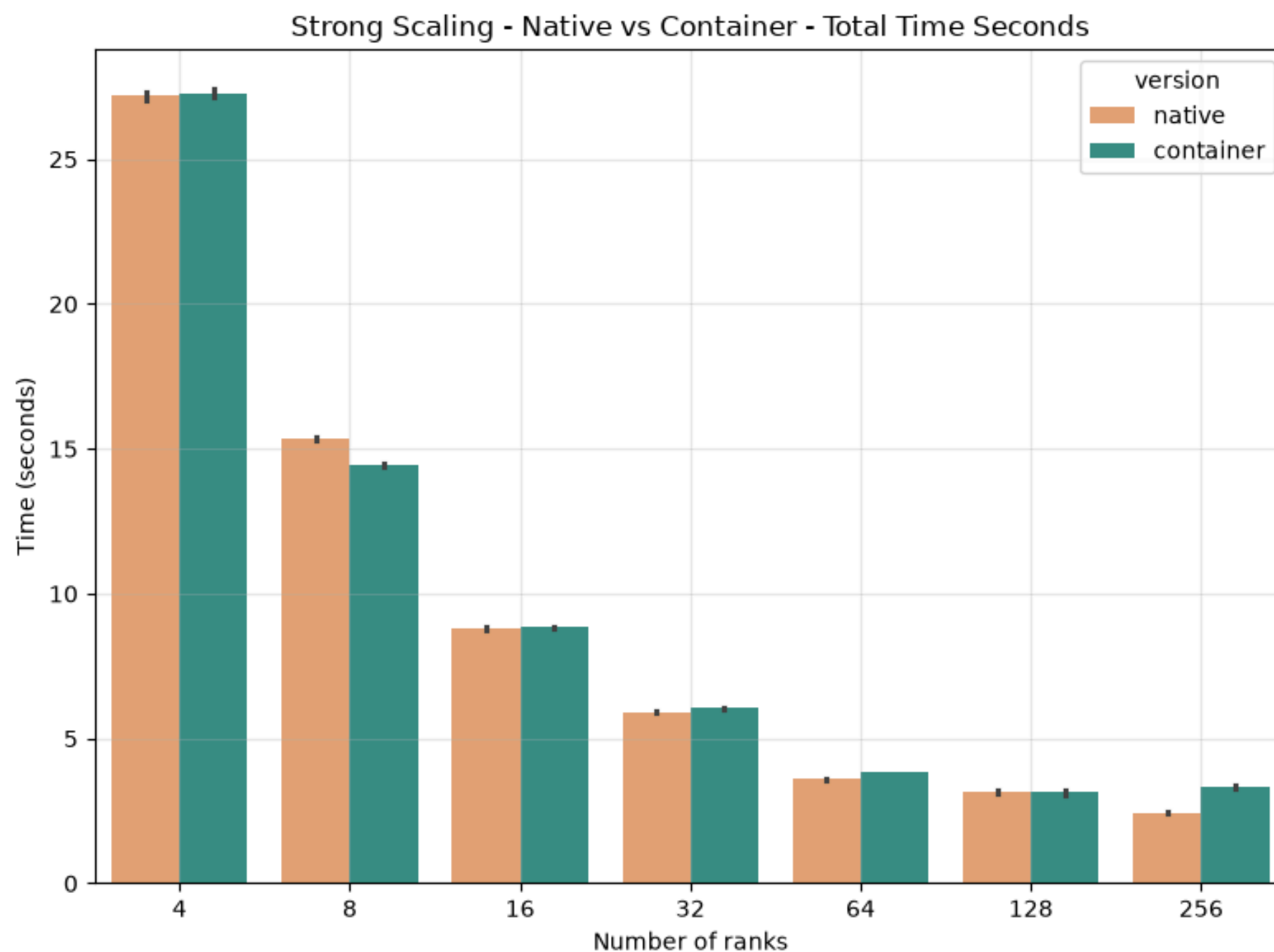


$$E_N \approx \frac{T_4}{T_N} \times 100$$

\*All other experiments are shown in the report. Refere to it for detailed results. This is meant only as 15 minutes presentation.

# NATIVE - CONTAINER

## strong scaling



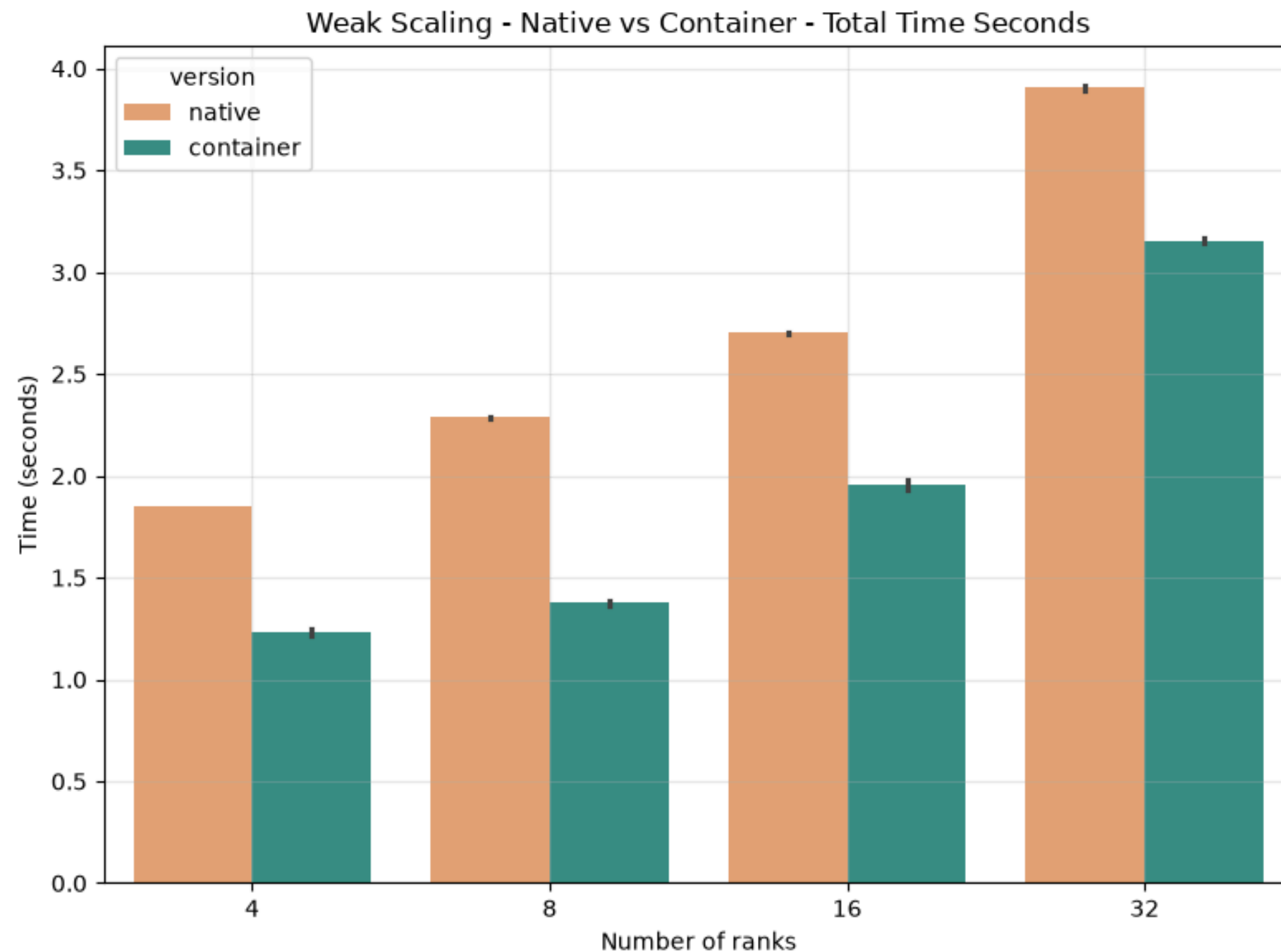
This is a nice comparable behavior. Looking at the other timings seems like everything is pretty balanced across the two different implementations. Notice, there is no OMP parallelization going on here.

\*All other experiments are shown in the report. Refere to it for detailed results. This is meant only as 15 minutes presentation.

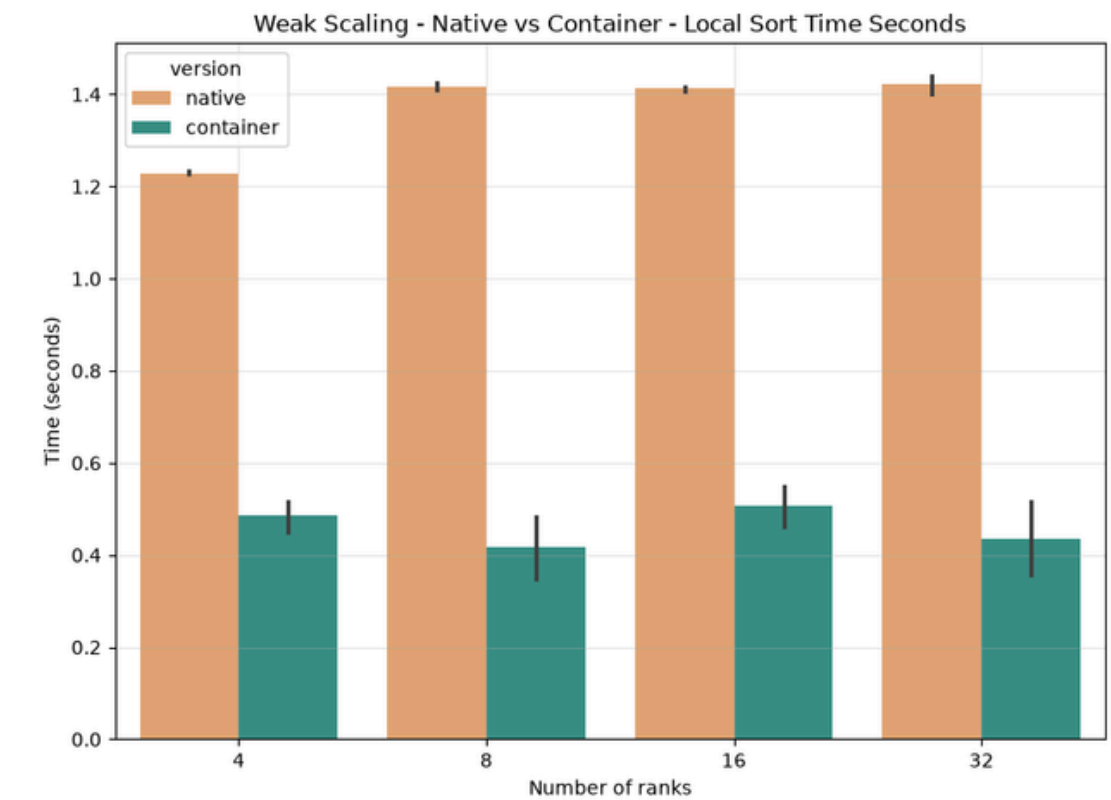
# NATIVE - CONTAINER

## weak scaling

22



this is almost surely caused by  
different OMP versions

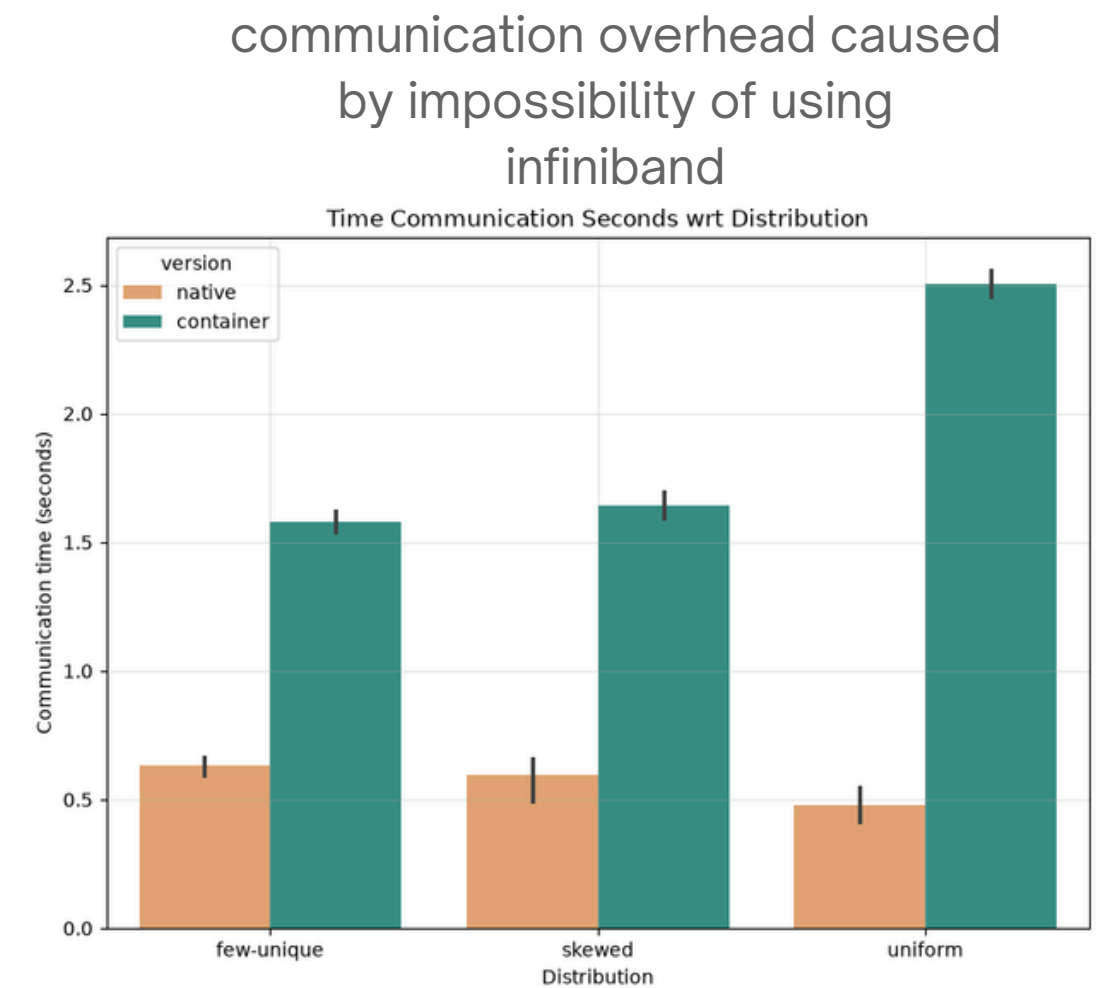
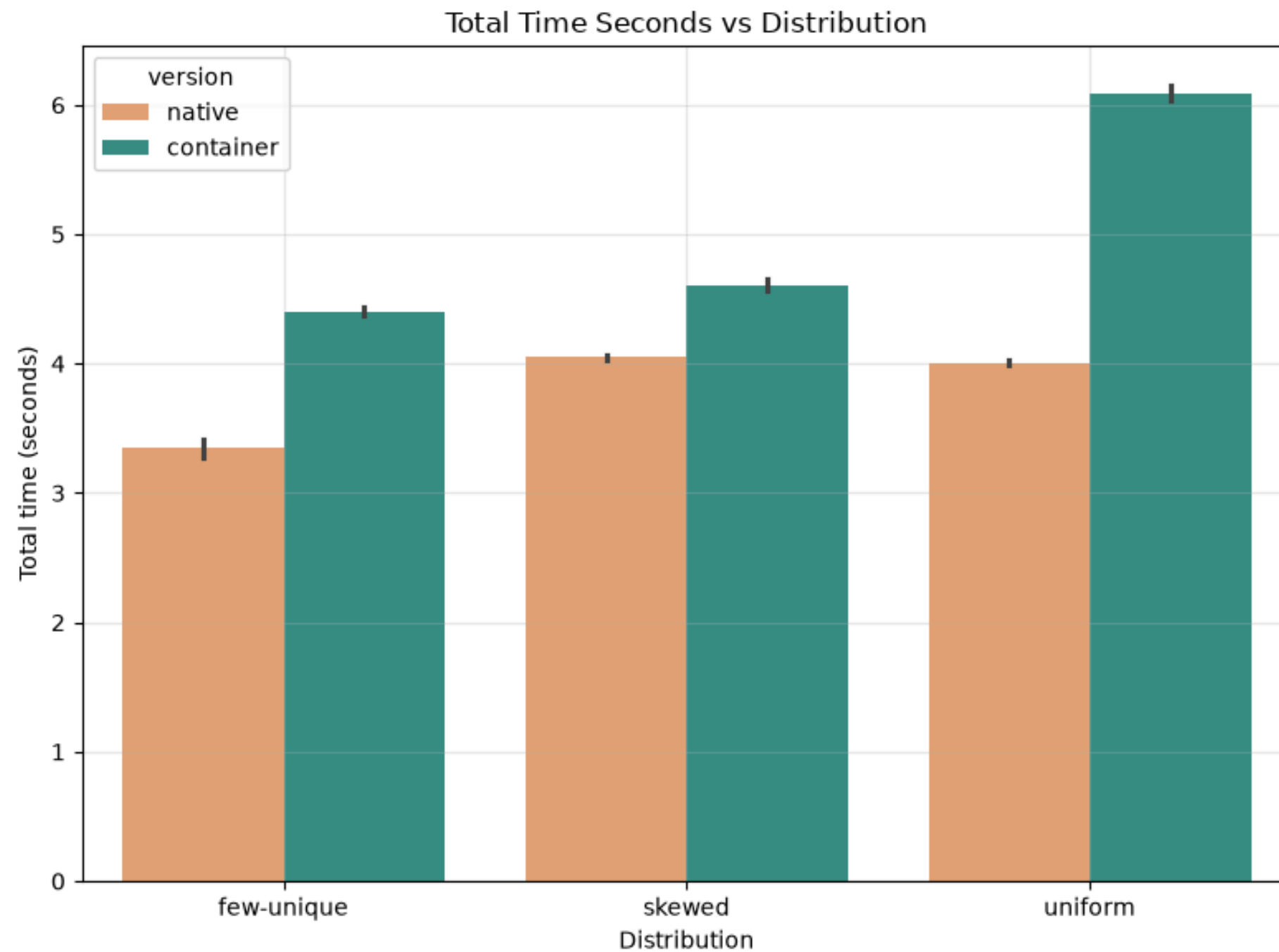


\*All other experiments are shown in the report. Refere to it for detailed results. This is meant only as 15 minutes presentation.

# CONTAINER - NATIVE

## skewed distribution

23

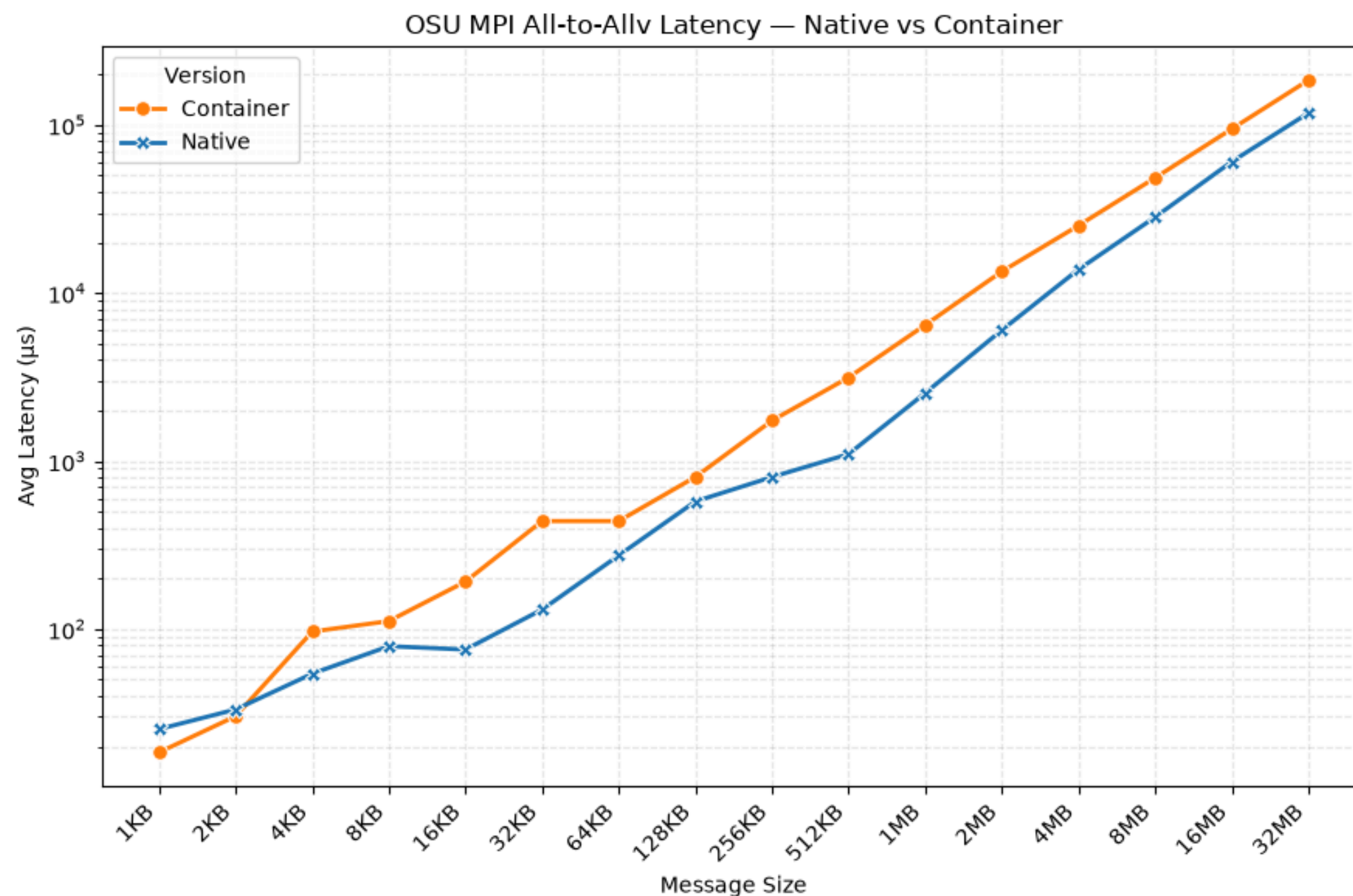


\*All other experiments are shown in the report. Refere to it for detailed results. This is meant only as 15 minutes presentation.

# OSU MICROBENCHMARK

## OSU Alltoallv

24



This confirm the communication overhead observed in the various experiments. This cannot tells anything about the decreased execution time.

\*All other experiments are shown in the report. Refere to it for detailed results. This is meant only as 15 minutes presentation.



**thank you for the  
attention**